

Chương 7 – Học Tập Tập Hợp và Rừng Ngẫu Nhiên

Sổ tay này chứa tất cả mã mẫu và giải pháp cho các bài tập trong chương 7.

 [Open in Colab](#)  [Open in Kaggle](#)

✓ Thiết lập

Dự án này yêu cầu Python 3.7 hoặc cao hơn.

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Như chúng ta đã làm trong các chương trước, hãy xác định kích thước phông chữ mặc định để làm cho các hình ảnh trở nên đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Và hãy tạo thư mục images/ensembles (nếu nó chưa tồn tại), và định nghĩa hàm save_fig() mà được sử dụng trong toàn bộ sổ tay này để lưu các hình ảnh ở độ phân giải cao cho sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "ensembles"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

✓ Các bộ phân loại theo hình thức bỏ phiếu

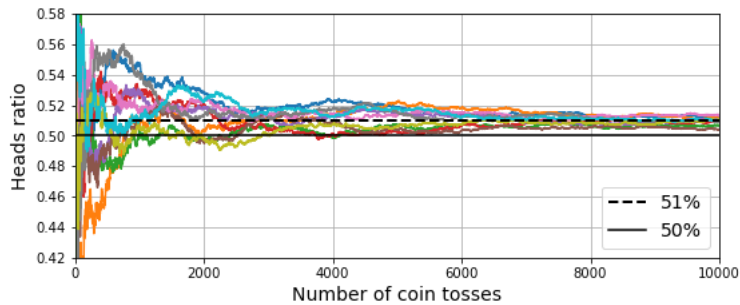
```
# extra code - this cell generates and saves Figure 7-3

import matplotlib.pyplot as plt
import numpy as np

heads_proba = 0.51
np.random.seed(42)
coin_tosses = (np.random.rand(10000, 10) < heads_proba).astype(np.int32)
cumulative_heads = coin_tosses.cumsum(axis=0)
cumulative_heads_ratio = cumulative_heads / np.arange(1, 10001).reshape(-1, 1)

plt.figure(figsize=(8, 3.5))
plt.plot(cumulative_heads_ratio)
plt.plot([0, 10000], [0.51, 0.51], "k--", linewidth=2, label="51%")
plt.plot([0, 10000], [0.5, 0.5], "k-", label="50%")
```

```
plt.xlabel("Number of coin tosses")
plt.ylabel("Heads ratio")
plt.legend(loc="lower right")
plt.axis([0, 10000, 0.42, 0.58])
plt.grid()
save_fig("law_of_large_numbers_plot")
plt.show()
```



Hãy xây dựng một bộ phân loại bỏ phiếu:

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)

VotingClassifier(estimators=[('lr', LogisticRegression(random_state=42)),
                             ('rf', RandomForestClassifier(random_state=42)),
                             ('svc', SVC(random_state=42))])
```

```
for name, clf in voting_clf.named_estimators_.items():
    print(name, "=", clf.score(X_test, y_test))
```

```
lr = 0.864
rf = 0.896
svc = 0.896
```

```
voting_clf.predict(X_test[:1])
```

```
array([1])
```

```
[clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
```

```
[array([1]), array([1]), array([0])]
```

```
voting_clf.score(X_test, y_test)
```

```
0.912
```

Bây giờ hãy sử dụng bỏ phiếu mềm:

```
voting_clf.voting = "soft"
voting_clf.named_estimators_["svc"].probability = True
voting_clf.fit(X_train, y_train)
voting_clf.score(X_test, y_test)
```

```
0.92
```

▼ Gợi và Dán

Bagging và Pasting trong Scikit-Learn

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
```

```
bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                           max_samples=100, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
```

```
BaggingClassifier(base_estimator=DecisionTreeClassifier(), max_samples=100,
                  n_estimators=500, n_jobs=-1, random_state=42)
```

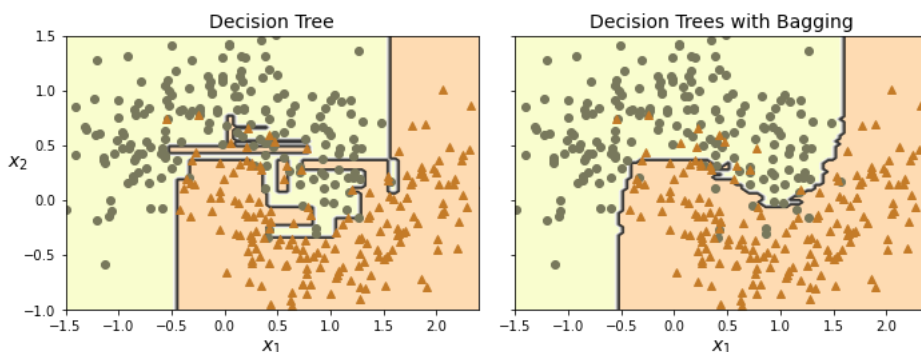
extra code - this cell generates and saves Figure 7-5

```
def plot_decision_boundary(clf, X, y, alpha=1.0):
    axes=[-1.5, 2.4, -1, 1.5]
    x1, x2 = np.meshgrid(np.linspace(axes[0], axes[1], 100),
                          np.linspace(axes[2], axes[3], 100))
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)

    plt.contourf(x1, x2, y_pred, alpha=0.3 * alpha, cmap='Wistia')
    plt.contour(x1, x2, y_pred, cmap="Greys", alpha=0.8 * alpha)
    colors = ["#78785c", "#c47b27"]
    markers = ("o", "^")
    for idx in (0, 1):
        plt.plot(X[:, 0][y == idx], X[:, 1][y == idx],
                 color=colors[idx], marker=markers[idx], linestyle="none")
    plt.axis(axes)
    plt.xlabel(r"$x_1$")
    plt.ylabel(r"$x_2$", rotation=0)
```

```
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
```

```
fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)
plt.sca(axes[0])
plot_decision_boundary(tree_clf, X_train, y_train)
plt.title("Decision Tree")
plt.sca(axes[1])
plot_decision_boundary(bag_clf, X_train, y_train)
plt.title("Decision Trees with Bagging")
plt.ylabel("")
save_fig("decision_tree_without_and_with_bagging_plot")
plt.show()
```



▼ Cách đánh giá Out-of-Bag

```
bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
                           oob_score=True, n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
bag_clf.oob_score_
```

```
0.896
```

```
bag_clf.oob_decision_function_[:3] # probas for the first 3 instances
```

```
array([[0.32352941, 0.67647059],
       [0.3375    , 0.6625    ],
       [1.        , 0.        ]])
```

```
from sklearn.metrics import accuracy_score
```

```
y_pred = bag_clf.predict(X_test)
accuracy_score(y_test, y_pred)
```

```
0.92
```

Dưới đây là bản dịch sang tiếng Việt:

Nếu bạn rút ngẫu nhiên một bản sao từ một tập dữ liệu có kích thước m , mỗi bản sao trong tập dữ liệu rõ ràng có xác suất $1/m$ để được chọn, và do đó nó có xác suất $1 - 1/m$ để không được chọn. Nếu bạn rút m bản sao có hoàn lại, tất cả các lần rút đều độc lập và do đó mỗi bản sao có xác suất $(1 - 1/m)^m$ để không được chọn. Bây giờ hãy sử dụng thực tế rằng $\exp(x)$ bằng giới hạn của $(1 + x/m)^m$ khi m tiến tới vô cùng. Vậy nếu m lớn, tỷ lệ các bản sao không được chọn sẽ khoảng $\exp(-1) \approx 0,37$. Vì vậy, khoảng 63% ($1 - 0,37$) sẽ được lấy mẫu.

```
# extra code - shows how to compute the 63% proba
print(1 - (1 - 1 / 1000) ** 1000)
print(1 - np.exp(-1))
```

```
0.6323045752290363
0.6321205588285577
```

✓ Rừng Ngẫu Nhiên

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
                                n_jobs=-1, random_state=42)

rnd_clf.fit(X_train, y_train)
y_pred_rf = rnd_clf.predict(X_test)
```

Cây quyết định ngẫu nhiên tương đương với một túi các cây quyết định:

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),
    n_estimators=500, n_jobs=-1, random_state=42)
```

```
# extra code - verifies that the predictions are identical
bag_clf.fit(X_train, y_train)
y_pred_bag = bag_clf.predict(X_test)
np.all(y_pred_bag == y_pred_rf) # same predictions
```

```
True
```

✓ Các yếu tố quan trọng

```
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
rnd_clf.fit(iris.data, iris.target)
for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
    print(round(score, 2), name)
```

```
0.11 sepal length (cm)
0.02 sepal width (cm)
0.44 petal length (cm)
0.42 petal width (cm)
```

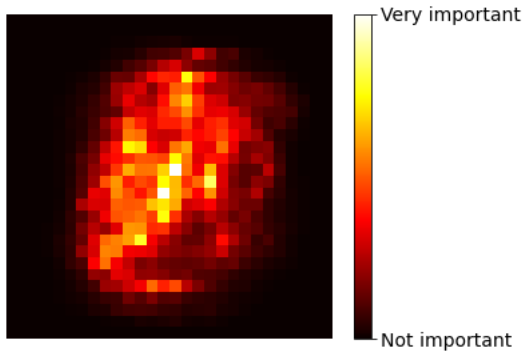
```
# extra code - this cell generates and saves Figure 7-6

from sklearn.datasets import fetch_openml

X_mnist, y_mnist = fetch_openml('mnist_784', return_X_y=True, as_frame=False,
                                parser='auto')

rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rnd_clf.fit(X_mnist, y_mnist)

heatmap_image = rnd_clf.feature_importances_.reshape(28, 28)
plt.imshow(heatmap_image, cmap="hot")
cbar = plt.colorbar(ticks=[rnd_clf.feature_importances_.min(),
                           rnd_clf.feature_importances_.max()])
cbar.ax.set_yticklabels(['Not important', 'Very important'], fontsize=14)
plt.axis("off")
save_fig("mnist_feature_importance_plot")
plt.show()
```



▼ Tăng cường

Tăng cường Ada (AdaBoost)

```
# extra code - this cell generates and saves Figure 7-8

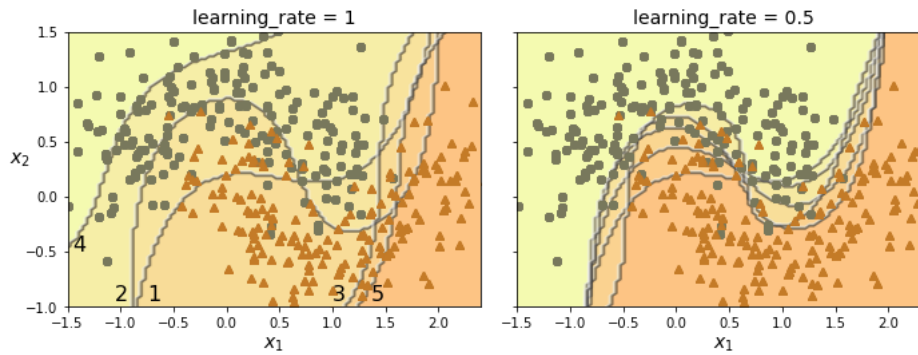
m = len(X_train)

fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)
for subplot, learning_rate in ((0, 1), (1, 0.5)):
    sample_weights = np.ones(m) / m
    plt.sca(axes[subplot])
    for i in range(5):
        svm_clf = SVC(C=0.2, gamma=0.6, random_state=42)
        svm_clf.fit(X_train, y_train, sample_weight=sample_weights * m)
        y_pred = svm_clf.predict(X_train)

        error_weights = sample_weights[y_pred != y_train].sum()
        r = error_weights / sample_weights.sum() # equation 7-1
        alpha = learning_rate * np.log((1 - r) / r) # equation 7-2
        sample_weights[y_pred != y_train] *= np.exp(alpha) # equation 7-3
        sample_weights /= sample_weights.sum() # normalization step

    plot_decision_boundary(svm_clf, X_train, y_train, alpha=0.4)
    plt.title(f"learning_rate = {learning_rate}")
    if subplot == 0:
        plt.text(-0.75, -0.95, "1", fontsize=16)
        plt.text(-1.05, -0.95, "2", fontsize=16)
        plt.text(1.0, -0.95, "3", fontsize=16)
        plt.text(-1.45, -0.5, "4", fontsize=16)
        plt.text(1.36, -0.95, "5", fontsize=16)
    else:
        plt.ylabel("")

save_fig("boosting_plot")
plt.show()
```

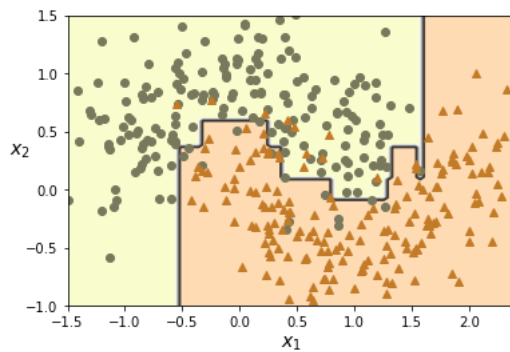


```
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42)
ada_clf.fit(X_train, y_train)

AdaBoostClassifier(base_estimator=DecisionTreeClassifier(max_depth=1),
                    learning_rate=0.5, n_estimators=30, random_state=42)
```

```
# extra code - in case you're curious to see what the decision boundary
# looks like for the AdaBoost classifier
plot_decision_boundary(ada_clf, X_train, y_train)
```



✓ Tăng cường Gradient

Hãy tạo một tập dữ liệu quadratic đơn giản và gán một DecisionTreeRegressor cho nó:

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100) # y = 3x^2 + Gaussian noise

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)

DecisionTreeRegressor(max_depth=2, random_state=42)
```

Bây giờ hãy huấn luyện một cây quyết định hồi quy khác trên các sai số còn lại do trình dự đoán trước đó gây ra:

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)

DecisionTreeRegressor(max_depth=2, random_state=43)
```

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)
```

```
tree_reg3.fit(X, y3)
```

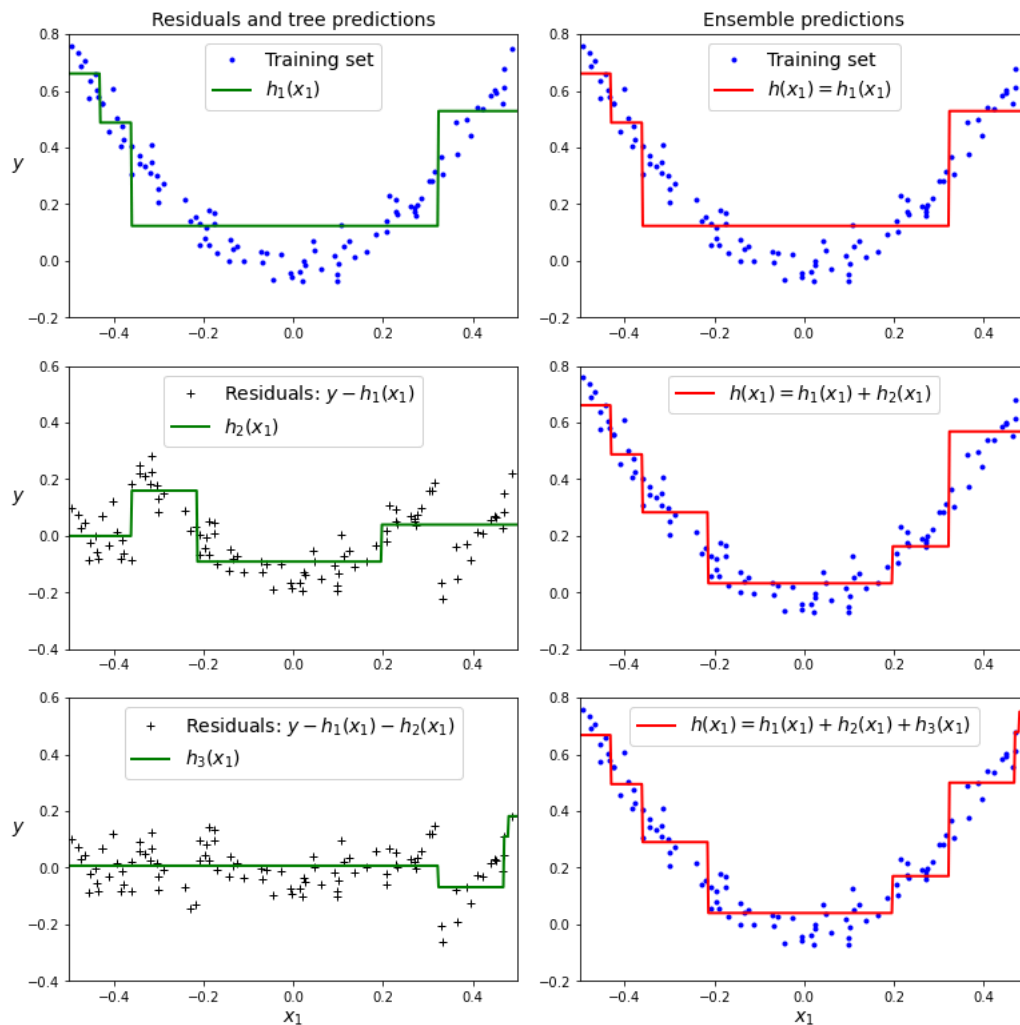
```
DecisionTreeRegressor(max_depth=2, random_state=44)
```

```
X_new = np.array([[ -0.4], [0.], [0.5]])  
sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
```

```
array([0.49484029, 0.04021166, 0.75026781])
```

```
# extra code - this cell generates and saves Figure 7-9
```

```
def plot_predictions(regressors, X, y, axes, style,  
                    label=None, data_style="b.", data_label=None):  
    x1 = np.linspace(axes[0], axes[1], 500)  
    y_pred = sum(regressor.predict(x1.reshape(-1, 1))  
                for regressor in regressors)  
    plt.plot(X[:, 0], y, data_style, label=data_label)  
    plt.plot(x1, y_pred, style, linewidth=2, label=label)  
    if label or data_label:  
        plt.legend(loc="upper center")  
    plt.axis(axes)  
  
plt.figure(figsize=(11, 11))  
  
plt.subplot(3, 2, 1)  
plot_predictions([tree_reg1], X, y, axes=[-0.5, 0.5, -0.2, 0.8], style="g-",  
                label="$h_1(x_1)$", data_label="Training set")  
plt.ylabel("$y$ ", rotation=0)  
plt.title("Residuals and tree predictions")  
  
plt.subplot(3, 2, 2)  
plot_predictions([tree_reg1], X, y, axes=[-0.5, 0.5, -0.2, 0.8], style="r-",  
                label="$h(x_1) = h_1(x_1)$", data_label="Training set")  
plt.title("Ensemble predictions")  
  
plt.subplot(3, 2, 3)  
plot_predictions([tree_reg2], X, y2, axes=[-0.5, 0.5, -0.4, 0.6], style="g-",  
                label="$h_2(x_1)$", data_style="k+",  
                data_label="Residuals: $y - h_1(x_1)$")  
plt.ylabel("$y$ ", rotation=0)  
  
plt.subplot(3, 2, 4)  
plot_predictions([tree_reg1, tree_reg2], X, y, axes=[-0.5, 0.5, -0.2, 0.8],  
                style="r-", label="$h(x_1) = h_1(x_1) + h_2(x_1)$")  
  
plt.subplot(3, 2, 5)  
plot_predictions([tree_reg3], X, y3, axes=[-0.5, 0.5, -0.4, 0.6], style="g-",  
                label="$h_3(x_1)$", data_style="k+",  
                data_label="Residuals: $y - h_1(x_1) - h_2(x_1)$")  
plt.xlabel("$x_1$")  
plt.ylabel("$y$ ", rotation=0)  
  
plt.subplot(3, 2, 6)  
plot_predictions([tree_reg1, tree_reg2, tree_reg3], X, y,  
                axes=[-0.5, 0.5, -0.2, 0.8], style="r-",  
                label="$h(x_1) = h_1(x_1) + h_2(x_1) + h_3(x_1)$")  
plt.xlabel("$x_1$")  
  
save_fig("gradient_boosting_plot")  
plt.show()
```



Bây giờ chúng ta hãy thử một bộ hồi quy gradient boosting:

```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                learning_rate=1.0, random_state=42)

gbrt.fit(X, y)
```

```
GradientBoostingRegressor(learning_rate=1.0, max_depth=2, n_estimators=3,
                           random_state=42)
```

```
gbrt_best = GradientBoostingRegressor(
    max_depth=2, learning_rate=0.05, n_estimators=500,
    n_iter_no_change=10, random_state=42)
gbrt_best.fit(X, y)
```

```
GradientBoostingRegressor(learning_rate=0.05, max_depth=2, n_estimators=500,
                           n_iter_no_change=10, random_state=42)
```

```
gbrt_best.n_estimators_
```

```
92
```

extra code - this cell generates and saves Figure 7-10

```
fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)

plt.sca(axes[0])
plot_predictions([gbrt], X, y, axes=[-0.5, 0.5, -0.1, 0.8], style="r-",
                 label="Ensemble predictions")
plt.title(f"learning_rate={gbrt.learning_rate}, "
          f"n_estimators={gbrt.n_estimators_}")
```



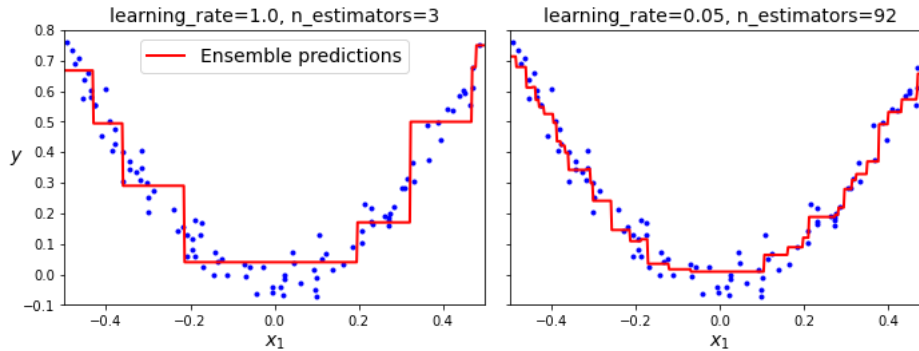
```

plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)

plt.sca(axes[1])
plot_predictions([gbrt_best], X, y, axes=[-0.5, 0.5, -0.1, 0.8], style="r-")
plt.title(f"learning_rate={gbrt_best.learning_rate}, "
         f"n_estimators={gbrt_best.n_estimators}")
plt.xlabel("$x_1$")

save_fig("gbrt_learning_rate_plot")
plt.show()

```



extra code - at least not in this chapter, it's presented in chapter 2

```

import pandas as pd
from sklearn.model_selection import train_test_split
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
            housing_tarball.extractall(path="datasets")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing = load_housing_data()

train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
housing_labels = train_set["median_house_value"]
housing = train_set.drop("median_house_value", axis=1)

```

```

from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.preprocessing import OrdinalEncoder

hgb_reg = make_pipeline(
    make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
                           remainder="passthrough"),
    HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
)
hgb_reg.fit(housing, housing_labels)

Pipeline(steps=[('columntransformer',
                 ColumnTransformer(remainder='passthrough',
                                   transformers=[('ordinalencoder',
                                                  OrdinalEncoder(),
                                                  ['ocean_proximity'])])),
                ('histgradientboostingregressor',
                 HistGradientBoostingRegressor(categorical_features=[0],
                                                random_state=42))])

```

extra code - evaluate the RMSE stats for the hgb_reg model

```

from sklearn.model_selection import cross_val_score

hgb_rmse = -cross_val_score(hgb_reg, housing, housing_labels,

```

```
scoring="neg_root_mean_squared_error", cv=10)
pd.Series(hgb_rmses).describe()
```

```
count      10.000000
mean      47613.307194
std       1295.422509
min       44963.213061
25%      47001.233485
50%      48000.963564
75%      48488.093243
max       49176.368465
dtype: float64
```

✧ Xếp chồng (Stacking)

```
from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5 # number of cross-validation folds
)
stacking_clf.fit(X_train, y_train)
```

```
StackingClassifier(cv=5,
                   estimators=[('lr', LogisticRegression(random_state=42)),
                                ('rf', RandomForestClassifier(random_state=42)),
                                ('svc', SVC(probability=True, random_state=42))],
                   final_estimator=RandomForestClassifier(random_state=43))
```

```
stacking_clf.score(X_test, y_test)
```

```
0.928
```

✧ Hướng dẫn bài tập

✧ 1. tới 7.

1. Nếu bạn đã huấn luyện năm mô hình khác nhau và tất cả đều đạt độ chính xác 95%, bạn có thể thử kết hợp chúng thành một bộ voting ensemble, điều này thường sẽ mang lại kết quả tốt hơn. Nó hoạt động hiệu quả hơn nếu các mô hình rất khác nhau (ví dụ, một bộ phân loại SVM, một bộ phân loại Cây Quyết Định, một bộ phân loại hồi quy logistic, v.v.). Càng tốt hơn nếu chúng được huấn luyện trên các tập huấn luyện khác nhau (đó là lý do của các bộ bagging và pasting), nhưng nếu không, điều này vẫn sẽ hiệu quả miễn là các mô hình rất khác nhau.
2. Một bộ phân loại voting cứng chỉ đếm số phiếu của mỗi bộ phân loại trong ensemble và chọn lớp nhận được nhiều phiếu nhất. Một bộ phân loại voting mềm tính toán trung bình xác suất lớp ước lượng cho mỗi lớp và chọn lớp có xác suất cao nhất. Điều này mang lại trọng số lớn hơn cho các phiếu bầu có độ tin cậy cao và thường hoạt động tốt hơn, nhưng chỉ hiệu quả nếu mỗi bộ phân loại đều có khả năng ước lượng xác suất lớp (ví dụ, đối với các bộ phân loại SVM trong Scikit-Learn, bạn phải đặt probability=True).
3. Có khả năng làm nhanh quá trình huấn luyện của một ensemble bagging bằng cách phân phối nó qua nhiều máy chủ, vì mỗi bộ dự đoán trong ensemble độc lập với các bộ khác. Điều này cũng đúng với các ensemble pasting và Random Forests, vì lý do tương tự. Tuy nhiên, mỗi bộ dự đoán trong một ensemble boosting được xây dựng dựa trên bộ dự đoán trước đó, vì vậy quá trình huấn luyện là tuần tự và bạn sẽ không có được lợi ích gì khi phân phối huấn luyện qua nhiều máy chủ.
4. Về các ensemble stacking, tất cả các bộ dự đoán trong một lớp nhất định độc lập với nhau, vì vậy chúng có thể được huấn luyện song song trên nhiều máy chủ. Tuy nhiên, các bộ dự đoán trong một lớp chỉ có thể được huấn luyện sau khi các bộ dự đoán trong lớp trước đó đã được huấn luyện hoàn tất.
5. Với đánh giá out-of-bag, mỗi bộ dự đoán trong một ensemble bagging được đánh giá bằng cách sử dụng các trường hợp mà nó chưa được huấn luyện (chúng đã được giữ lại). Điều này cho phép bạn có một đánh giá tương đối không thiên lệch về ensemble mà không cần một tập dữ liệu xác thực bổ sung. Do đó, bạn có nhiều trường hợp hơn để huấn luyện, và ensemble của bạn có thể hoạt động tốt hơn một chút.

6. Khi bạn đang phát triển một cây trong Random Forest, chỉ một tập hợp ngẫu nhiên các đặc trưng được xem xét để phân chia tại mỗi nút. Điều này cũng đúng với Extra-Trees, nhưng chúng đi một bước xa hơn: thay vì tìm kiếm các ngưỡng tốt nhất có thể, như các Cây Quyết Định thông thường làm, chúng sử dụng các ngưỡng ngẫu nhiên cho mỗi đặc trưng. Sự ngẫu nhiên bổ sung này hoạt động như một hình thức regularization: nếu một Random Forest đang overfit dữ liệu huấn luyện, Extra-Trees có thể hoạt động tốt hơn. Hơn nữa, vì Extra-Trees không tìm kiếm các ngưỡng tốt nhất có thể, chúng nhanh hơn rất nhiều trong việc huấn luyện so với Random Forests. Tuy nhiên, chúng không nhanh hơn cũng không chậm hơn Random Forests khi đưa ra dự đoán.
7. Nếu ensemble AdaBoost của bạn underfit dữ liệu huấn luyện, bạn có thể thử tăng số lượng bộ phân loại hoặc giảm các tham số điều chỉnh hyperparameter của bộ phân loại cơ bản. Bạn cũng có thể thử tăng nhẹ tốc độ học. Nếu ensemble Gradient Boosting của bạn overfit tập huấn luyện, bạn nên thử giảm tốc độ học. Bạn cũng có thể sử dụng early stopping để tìm ra số lượng bộ phân loại phù hợp (có thể bạn có quá nhiều).

8. Bộ phân loại bỏ phiếu

Bài tập: Tải dữ liệu MNIST và chia nó thành tập huấn luyện, tập xác thực và tập kiểm tra (ví dụ: sử dụng 50.000 mẫu cho tập huấn luyện, 10.000 mẫu cho tập xác thực và 10.000 mẫu cho tập kiểm tra).

Bộ dữ liệu MNIST đã được tải lên trước đó. Bộ dữ liệu đã được chia thành một tập huấn luyện (60.000 mẫu đầu tiên) và một tập kiểm tra (10.000 mẫu cuối cùng), và tập huấn luyện đã được xáo trộn trước. Vậy nên, những gì chúng ta cần làm là lấy 50.000 mẫu đầu tiên cho tập huấn luyện mới, 10.000 mẫu tiếp theo cho tập xác thực, và 10.000 mẫu cuối cùng cho tập kiểm tra:

```
X_train, y_train = X_mnist[:50_000], y_mnist[:50_000]
X_valid, y_valid = X_mnist[50_000:60_000], y_mnist[50_000:60_000]
X_test, y_test = X_mnist[60_000:], y_mnist[60_000:]
```

Bài tập: Sau đó, huấn luyện các bộ phân loại khác nhau, chẳng hạn như bộ phân loại Random Forest, bộ phân loại Extra-Trees và SVM.

```
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.svm import LinearSVC
from sklearn.neural_network import MLPClassifier
```

Chú ý: LinearSVC có một siêu tham số bậc hai mà giá trị mặc định sẽ thay đổi từ True thành "auto" trong Scikit-Learn 1.5. Để đảm bảo rằng tài liệu này tiếp tục sản xuất các đầu ra giống nhau, tôi sẽ đặt nó một cách rõ ràng thành True. Vui lòng xem tài liệu để biết thêm chi tiết.

```
random_forest_clf = RandomForestClassifier(n_estimators=100, random_state=42)
extra_trees_clf = ExtraTreesClassifier(n_estimators=100, random_state=42)
svm_clf = LinearSVC(max_iter=100, tol=20, dual=True, random_state=42)
mlp_clf = MLPClassifier(random_state=42)
```

```
estimators = [random_forest_clf, extra_trees_clf, svm_clf, mlp_clf]
for estimator in estimators:
    print("Training the", estimator)
    estimator.fit(X_train, y_train)
```

```
Training the RandomForestClassifier(random_state=42)
Training the ExtraTreesClassifier(random_state=42)
Training the LinearSVC(max_iter=100, random_state=42, tol=20)
Training the MLPClassifier(random_state=42)
```

```
[estimator.score(X_valid, y_valid) for estimator in estimators]
```

```
[0.9736, 0.9743, 0.8662, 0.966]
```

Linear SVM bị các bộ phân loại khác vượt trội hơn hẳn. Tuy nhiên, hãy giữ lại nó cho bây giờ vì nó có thể cải thiện hiệu suất của bộ phân loại voting.

Bài tập: Tiếp theo, hãy thử kết hợp [các bộ phân loại] thành một tập hợp (ensemble) mà có hiệu suất cao hơn tất cả trên tập xác thực, sử dụng bộ phân loại bỏ phiếu mềm (soft voting) hoặc bỏ phiếu cứng (hard voting).

```
from sklearn.ensemble import VotingClassifier
```

```
named_estimators = [  
    ("random_forest_clf", random_forest_clf),  
    ("extra_trees_clf", extra_trees_clf),  
    ("svm_clf", svm_clf),  
    ("mlp_clf", mlp_clf),  
]
```

```
voting_clf = VotingClassifier(named_estimators)
```

```
voting_clf.fit(X_train, y_train)
```

```
VotingClassifier(estimators=[('random_forest_clf',  
                             RandomForestClassifier(random_state=42)),  
                             ('extra_trees_clf',  
                              ExtraTreesClassifier(random_state=42)),  
                             ('svm_clf',  
                              LinearSVC(max_iter=100, random_state=42, tol=20)),  
                             ('mlp_clf', MLPClassifier(random_state=42))])
```

```
voting_clf.score(X_valid, y_valid)
```

```
0.9758
```

VotingClassifier đã tạo một bản sao của mỗi bộ phân loại và nó đã huấn luyện các bản sao đó bằng các chỉ số lớp như là nhãn, không phải tên lớp gốc. Do đó, để đánh giá các bản sao này, chúng ta cần cung cấp các chỉ số lớp. Để chuyển đổi các lớp thành các chỉ số lớp, chúng ta có thể sử dụng LabelEncoder.

```
from sklearn.preprocessing import LabelEncoder  
  
encoder = LabelEncoder()  
y_valid_encoded = encoder.fit_transform(y_valid)
```

Tuy nhiên, trong trường hợp của MNIST, việc chuyển đổi tên lớp thành số nguyên dễ hơn, vì các chữ số khớp với các id lớp.

```
y_valid_encoded = y_valid.astype(np.int64)
```

Bây giờ hãy đánh giá các bản sao của bộ phân loại:

```
[estimator.score(X_valid, y_valid_encoded)  
 for estimator in voting_clf.estimators_]
```

```
[0.9736, 0.9743, 0.8662, 0.966]
```

Hãy loại bỏ SVM để xem hiệu suất có cải thiện không. Có thể loại bỏ một bộ ước lượng bằng cách đặt nó thành 'drop' bằng cách sử dụng set_params() như sau:

```
voting_clf.set_params(svm_clf="drop")
```

```
VotingClassifier(estimators=[('random_forest_clf',  
                             RandomForestClassifier(random_state=42)),  
                             ('extra_trees_clf',  
                              ExtraTreesClassifier(random_state=42)),  
                             ('svm_clf', 'drop'),  
                             ('mlp_clf', MLPClassifier(random_state=42))])
```

Điều này đã cập nhật danh sách các ước lượng.

```
voting_clf.estimators
```

```
[('random_forest_clf', RandomForestClassifier(random_state=42)),  
 ('extra_trees_clf', ExtraTreesClassifier(random_state=42)),  
 ('svm_clf', 'drop'),  
 ('mlp_clf', MLPClassifier(random_state=42))]
```

Bắt đầu lập trình hoặc [tạo](#) mã bằng trí tuệ nhân tạo (AI).

Tuy nhiên, nó đã không cập nhật danh sách các bộ ước lượng đã được huấn luyện/đã được đào tạo:

```
voting_clf.estimators_
```

```
[RandomForestClassifier(random_state=42),  
 ExtraTreesClassifier(random_state=42),  
 LinearSVC(max_iter=100, random_state=42, tol=20),  
 MLPClassifier(random_state=42)]
```

```
voting_clf.named_estimators_
```

```
{'random_forest_clf': RandomForestClassifier(random_state=42),  
 'extra_trees_clf': ExtraTreesClassifier(random_state=42),  
 'svm_clf': LinearSVC(max_iter=100, random_state=42, tol=20),  
 'mlp_clf': MLPClassifier(random_state=42)}
```

Vì vậy, chúng ta có thể hoặc là huấn luyện lại (fit) Bộ phân loại Bầu chọn (VotingClassifier), hoặc chỉ cần loại bỏ SVM khỏi danh sách các bộ ước lượng đã được huấn luyện, ở cả hai vị trí là `estimators_` và `named_estimators_`:

```
svm_clf_trained = voting_clf.named_estimators_.pop("svm_clf")  
voting_clf.estimators_.remove(svm_clf_trained)
```

Now let's evaluate the `VotingClassifier` again:

```
voting_clf.score(X_valid, y_valid)
```

```
0.9769
```

Tốt hơn một chút rồi! SVM đã làm tổn hại đến hiệu suất. Bây giờ chúng ta hãy thử sử dụng một bộ phân loại bầu chọn mềm (soft voting classifier). Chúng ta thực sự không cần phải huấn luyện lại bộ phân loại; chúng ta chỉ cần đặt thuộc tính voting thành "soft":

```
voting_clf.voting = "soft"
```

```
voting_clf.score(X_valid, y_valid)
```

```
0.9724
```

Không, bầu chọn cứng (hard voting) thắng trong trường hợp này.

Khi bạn đã tìm thấy [một mô hình tổ hợp/tổng hợp (ensemble) hoạt động tốt hơn các bộ dự đoán riêng lẻ], hãy thử nó trên tập kiểm thử (test set). Nó hoạt động tốt hơn bao nhiêu so với các bộ phân loại riêng lẻ?

```
voting_clf.voting = "hard"  
voting_clf.score(X_test, y_test)
```

```
0.9727
```

```
[estimator.score(X_test, y_test.astype(np.int64))  
 for estimator in voting_clf.estimators_]
```

```
[0.968, 0.9703, 0.965]
```

Bộ phân loại bầu chọn (voting classifier) đã giảm tỉ lệ lỗi (error rate) của mô hình tốt nhất từ khoảng 3% xuống còn 2.7%, điều này có nghĩa là giảm được 10% số lỗi.

✓ 9. Tổ hợp Xếp chồng (Stacking Ensemble)

Bài tập: Chạy các bộ phân loại riêng lẻ từ bài tập trước để đưa ra dự đoán trên tập kiểm định (validation set), và tạo một tập huấn luyện mới với các dự đoán thu được: mỗi thể hiện huấn luyện (training instance) là một vectơ chứa tập hợp các dự đoán từ tất cả các bộ phân loại của bạn cho một hình ảnh, và mục tiêu (target) là lớp của hình ảnh đó. Huấn luyện một bộ phân loại trên tập huấn luyện mới này.

```
X_valid_predictions = np.empty((len(X_valid), len(estimators)), dtype=object)
```

```
for index, estimator in enumerate(estimators):
    X_valid_predictions[:, index] = estimator.predict(X_valid)
```

X_valid_predictions

```
array([[ '3', '3', '3', '3'],
       [ '8', '8', '8', '8'],
       [ '6', '6', '6', '6'],
       ...,
       [ '5', '5', '5', '5'],
       [ '6', '6', '6', '6'],
       [ '8', '8', '8', '8']], dtype=object)
```

```
rnd_forest_blender = RandomForestClassifier(n_estimators=200, oob_score=True,
                                           random_state=42)
rnd_forest_blender.fit(X_valid_predictions, y_valid)
```

```
RandomForestClassifier(n_estimators=200, oob_score=True, random_state=42)
```

```
rnd_forest_blender.oob_score_
```

```
0.9722
```

Bạn có thể tinh chỉnh (fine-tune) bộ tổng hợp (blender) này hoặc thử các loại bộ tổng hợp khác (ví dụ: một MLPClassifier), sau đó chọn bộ tốt nhất bằng cách sử dụng kiểm định chéo (cross-validation), như thường lệ.

Bài tập: Chúc mừng, bạn vừa huấn luyện một bộ tổng hợp (blender), và cùng với các bộ phân loại, chúng tạo thành một tổ hợp xếp chồng (stacking ensemble)! Bây giờ chúng ta hãy đánh giá tổ hợp này trên tập kiểm thử (test set). Đối với mỗi hình ảnh trong tập kiểm thử, hãy thực hiện dự đoán bằng tất cả các bộ phân loại của bạn, sau đó đưa các dự đoán này vào bộ tổng hợp (blender) để nhận được dự đoán của tổ hợp. Kết quả so với bộ phân loại bầu chọn (voting classifier) bạn đã huấn luyện trước đó như thế nào?

```
X_test_predictions = np.empty((len(X_test), len(estimators)), dtype=object)
```

```
for index, estimator in enumerate(estimators):
    X_test_predictions[:, index] = estimator.predict(X_test)
```

```
y_pred = rnd_forest_blender.predict(X_test_predictions)
```

```
accuracy_score(y_test, y_pred)
```

```
0.9705
```

Tổ hợp xếp chồng này không hoạt động tốt bằng bộ phân loại bầu chọn mà chúng ta đã huấn luyện trước đó.

Bài tập: Bây giờ hãy thử lại bằng cách sử dụng StackingClassifier thay thế: bạn có đạt được hiệu suất tốt hơn không? Nếu có, tại sao?

Vì StackingClassifier sử dụng kiểm định chéo K-Fold (K-Fold cross-validation), chúng ta không cần một tập kiểm định (validation set) riêng biệt, vì vậy, hãy gộp tập huấn luyện và tập kiểm định thành một tập huấn luyện lớn hơn:

```
X_train_full, y_train_full = X_mnist[:60_000], y_mnist[:60_000]
```

Bây giờ hãy tạo và huấn luyện bộ phân loại xếp chồng trên tập huấn luyện đầy đủ:

Cảnh báo: ô lệnh sau đây sẽ mất khá nhiều thời gian để chạy (15-30 phút tùy thuộc vào phần cứng của bạn), vì nó sử dụng kiểm định chéo K-Fold với 5 lần chia (folds) theo mặc định. Nó sẽ huấn luyện 4 bộ phân loại, mỗi bộ 5 lần trên 80% tập huấn luyện đầy đủ để đưa ra dự đoán, cộng thêm một lần cuối cùng cho mỗi bộ trên toàn bộ tập huấn luyện, và cuối cùng nó sẽ huấn luyện mô hình cuối cùng trên các dự đoán đó. Tổng cộng là 25 mô hình cần huấn luyện!

```
stack_clf = StackingClassifier(named_estimators,
                              final_estimator=rnd_forest_blender)
stack_clf.fit(X_train_full, y_train_full)

StackingClassifier(estimators=[('random_forest_clf',
                               RandomForestClassifier(random_state=42)),
                              ('extra_trees_clf',
```

```
ExtraTreesClassifier(random_state=42)),
('svm_clf',
 LinearSVC(max_iter=100, random_state=42,
           tol=20)),
('mlp_clf', MLPClassifier(random_state=42))],
final_estimator=RandomForestClassifier(n_estimators=200,
                                       oob_score=True,
                                       random_state=42))
```

```
stack_clf.score(X_test, y_test)
```

```
0.9784
```

StackingClassifier có hiệu suất vượt trội đáng kể so với việc triển khai kỹ thuật xếp chồng tùy chỉnh mà chúng ta đã thử trước đó! Điều này chủ yếu là do hai lý do:

Vì chúng ta có thể khôi phục tập kiểm định (validation set), nên StackingClassifier đã được huấn luyện trên một tập dữ liệu lớn hơn.

Nó đã sử dụng `predict_proba()` nếu có, hoặc `decision_function()` nếu có, hoặc `predict()` nếu không có cả hai. Điều này đã cung cấp cho bộ tổng hợp (blender) các dữ liệu đầu vào tinh tế hơn để làm việc.

Và đó là tất cả cho hôm nay, xin chúc mừng bạn đã hoàn thành chương và các bài tập!